



МИНОБРНАУКИ РОССИИ

**федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)**

Основная образовательная программа 09.03.02

«Информационные системы и технологии»

**Отчет по дисциплине «Алгоритмы и структуры обработки данных»
по лабораторной работе №5**

Тема: «Деревья»

Проверил
к.т.н. доцент

Ковалев И.А.

подпись

Выполнила
студентка группы ИДБ-25-06

Якунина В.С.

подпись

Москва, 2026

Оглавление

Введение.....	3
Тексты программ.....	4
BST.....	4
Реализация типов обхода дерева.....	6
5 задач с Leetcode.....	8
Заключение.....	13

Введение

Изучить деревья и реализовать их на практике. Работа была выполнена на языке программирования Python.

Тексты программ

BST

В ходе выполнения лабораторной работы необходимо было реализовать бинарное дерево поиска. На рисунке 1 представлен тест программы.

```
class TreeNode:
    """Класс узла"""
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def insert(root, val):
    """Добавление узла"""
    if root is None:
        return TreeNode(val)
    if val < root.val:
        root.left = insert(root.left, val)
    elif val > root.val:
        root.right = insert(root.right, val)
    return root

def search(root, val):
    """Поиск узла"""
    if root is None or root.val == val:
        return root
    if val < root.val:
        return search(root.left, val)
    return search(root.right, val)

def find_min(root):
    """Минимум - самый левый узел"""
    current = root
    while current.left:
        current = current.left
    return current

def find_max(root):
    """Максимум - самый правый узел"""
    current = root
    while current.right:
        current = current.right
    return current
```

```

def delete(root, val):
    """Удаление узла"""
    if root is None:
        return None

    if val < root.val:
        root.left = delete(root.left, val)
    elif val > root.val:
        root.right = delete(root.right, val)
    else:
        # Случай 1: лист или один ребёнок
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left
        # Случай 2: два ребёнка → заменяем на min из правого поддерева
        successor = find_min(root.right)
        root.val = successor.val
        root.right = delete(root.right, successor.val)

    return root

def main():
    values = [10, 5, 15, 3, 7, 20]
    root = None
    for v in values:
        root = insert(root, v)

    node = search(root, 7)
    print(node.val if node else "Не найден")

    afterdelete = delete(root, 7)
    node = search(afterdelete, 7)
    print(node.val if node else "Не найден")

if __name__ == "__main__":
    main()

```

7
Не найден

Рисунок 1

Реализация типов обхода дерева

В лабораторной работе необходимо было реализовать 4 типа обхода дерева. Тест программы представлен на рисунке 2.

```
from collections import deque
```

```
class TreeNode:
```

```
    """Класс узла"""
```

```
    def __init__(self, val=0, left=None, right=None):
```

```
        self.val = val
```

```
        self.left = left
```

```
        self.right = right
```

```
def insert(root, val):
```

```
    """Добавление узла"""
```

```
    if root is None:
```

```
        return TreeNode(val)
```

```
    if val < root.val:
```

```
        root.left = insert(root.left, val)
```

```
    elif val > root.val:
```

```
        root.right = insert(root.right, val)
```

```
    return root
```

```
#1
```

```
def inorder(root):
```

```
    """Левый → Корень → Правый (даёт отсортированный порядок в  
BST!)"""
```

```
    if root is None:
```

```
        return []
```

```
    return inorder(root.left) + [root.val] + inorder(root.right)
```

```
#2
```

```
def preorder(root):
```

```
    """Корень → Левый → Правый (копирование / сериализация дерева)"""
```

```
    if root is None:
```

```
        return []
```

```
    return [root.val] + preorder(root.left) + preorder(root.right)
```

```
#3
```

```
def postorder(root):
```

```
    """Левый → Правый → Корень (удаление дерева / вычисление  
выражений)"""
```

```
    if root is None:
```

```
        return []
```

```
    return postorder(root.left) + postorder(root.right) + [root.val]
```

```
#4
```

```
def level_order(root):
```

```

"""Обход по уровням - очередь"""
if not root:
    return []
result = []
queue = deque([root])
while queue:
    level = []
    for _ in range(len(queue)):
        node = queue.popleft()
        level.append(node.val)
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
    result.append(level)
return result

def main():
    values = [10, 5, 15, 3, 7, 20]
    root = None
    for v in values:
        root = insert(root, v)

    print("1", inorder(root))
    print("2", preorder(root))
    print("3", postorder(root))
    print("4", level_order(root))

if __name__ == "__main__":
    main()

```

```

1 [3, 5, 7, 10, 15, 20]
2 [10, 5, 3, 7, 15, 20]
3 [3, 7, 5, 20, 15, 10]
4 [[10], [5, 15], [3, 7, 20]]

```

Рисунок 2

5 задач с Leetcode

В лабораторной работе необходимо было применить полученные знания о деревьях на практике, для чего дан был список задач. Нужно было решить 5 или больше из них. Решение этих задач с тестами:

```
from collections import deque

class TreeNode():
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def inTree(values):
    if not values or values[0] is None:
        return None

    root = TreeNode(values[0])
    queue = deque([root])
    i = 1

    while queue and i < len(values):
        node = queue.popleft()

        # Левый ребенок
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i])
            queue.append(node.left)
        i += 1

        # Правый ребенок
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i])
            queue.append(node.right)
        i += 1

    return root

def treeToList(root):
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        node = queue.popleft()
```



```

        if node:
            result.append(node.val)
            queue.append(node.left)
            queue.append(node.right)
        else:
            result.append(None)

    while result and result[-1] is None:
        result.pop()

    return result

#100. Same Tree
def isSameTree(p, q):
    if p is None and q is None:
        return True

    if p is None or q is None:
        return False

    if p.val == q.val:
        return isSameTree(p.left, q.left) and isSameTree(p.right,
q.right)

    return False

#104. Maximum Depth of Binary Tree
def maxDepth(root):
    if not root:
        return 0
    return 1 + max(maxDepth(root.left), maxDepth(root.right))

#226. Invert Binary Tree
def invertTree(root):
    if not root:
        return None
    root.left, root.right = invertTree(root.right),
invertTree(root.left)
    return root

#102. Binary Tree Level Order Traversal
def levelOrder(root):
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)
        current_level = []

```

```

        for _ in range(level_size):
            node = queue.popleft()
            current_level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

        result.append(current_level)

    return result

#98. Validate Binary Search Tree
def isValidBST(root):
    return validate(root, float('-inf'), float('inf'))

def validate(node, low, high):
    if not node:
        return True
    if not (low < node.val < high):
        return False
    return (validate(node.left, low, node.val) and
            validate(node.right, node.val, high))

#208. Implement Trie (Prefix Tree)
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True

    def search(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.is_end

    def startsWith(self, prefix):
        node = self.root

```

```

    for ch in prefix:
        if ch not in node.children:
            return False
        node = node.children[ch]
    return True

```

#297. Serialize and Deserialize Binary Tree

class Codec:

```

    def serialize(self, root):
        def dfs(node):
            if not node:
                vals.append("null")
                return
            vals.append(str(node.val))
            dfs(node.left)
            dfs(node.right)

```

```

        vals = []
        dfs(root)
        return ",".join(vals)

```

```

    def deserialize(self, data):
        def dfs():
            val = next(vals)
            if val == "null":
                return None
            node = TreeNode(int(val))
            node.left = dfs()
            node.right = dfs()
            return node

```

```

        vals = iter(data.split(","))
        return dfs()

```

```

def main():
    print("Пример 100. Same Tree")
    a1 = [1, 2, 3]
    b1 = [1, 2, 3]
    atree1, btree1 = inTree(a1), inTree(b1)
    print(a1, "и", b1, ":", isSameTree(atree1, btree1))

```

```

print()

```

```

print("Пример 104. Maximum Depth of Binary Tree")
root = [3, 9, 20, None, None, 15, 7]
rootTree = inTree(root)
print(root, ":", maxDepth(rootTree))

```

```

print()

```

```

print("Пример 226. Invert Binary Tree")
root = [2,1,3]

```

```

rootTree = inTree(root)
print(root, ":", invertTree(rootTree))

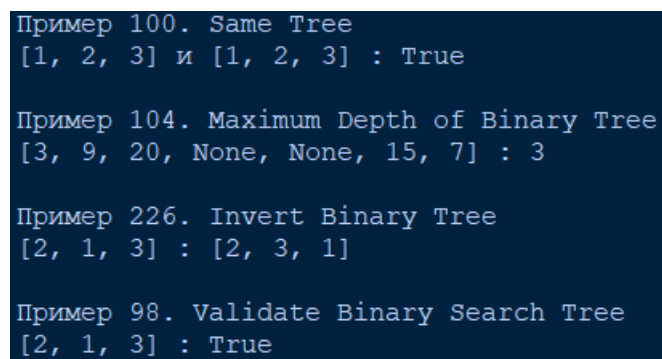
print()

print("Пример 98. Validate Binary Search Tree")
root = [2,1,3]
rootTree = inTree(root)
print(root, ":", isValidBST(rootTree))

if __name__ == "__main__":
    main()

```

Результаты тестирования программ представлены на рисунке 3.



```

Пример 100. Same Tree
[1, 2, 3] и [1, 2, 3] : True

Пример 104. Maximum Depth of Binary Tree
[3, 9, 20, None, None, 15, 7] : 3

Пример 226. Invert Binary Tree
[2, 1, 3] : [2, 3, 1]

Пример 98. Validate Binary Search Tree
[2, 1, 3] : True

```

Рисунок 3

Заключение

В результате выполнения лабораторной работы мы узнали, что такое деревья и использовали их на практике. Ссылка на [github](#).